

DAMA

Dual Agent Memory Architecture | Summary | 30 September 2026

DAMA does not try to give an AI model infinite context. It tries to make finite context intelligent.

1. What DAMA is

DAMA is a proposed AI architecture that splits one job into two. A powerful **Primary Agent** (a frontier LLM) does the thinking: reasoning, planning, coding and answering. A second, small and fast **Memory Agent** does the remembering: it decides what to store, what to fetch, what to update and what to forget. A **Memory Store** (vector database, knowledge graph, SQL, summaries) holds the data.

Part	Job	Type of model
Primary Agent	Thinks: reasons, plans, answers, uses tools	Large frontier LLM
Memory Agent	Remembers: stores, retrieves, updates, merges, compresses, forgets, sets a token budget	Small, fast, low-cost; returns structured decisions
Memory Store	Persists: holds the data	Databases and graphs, not a model

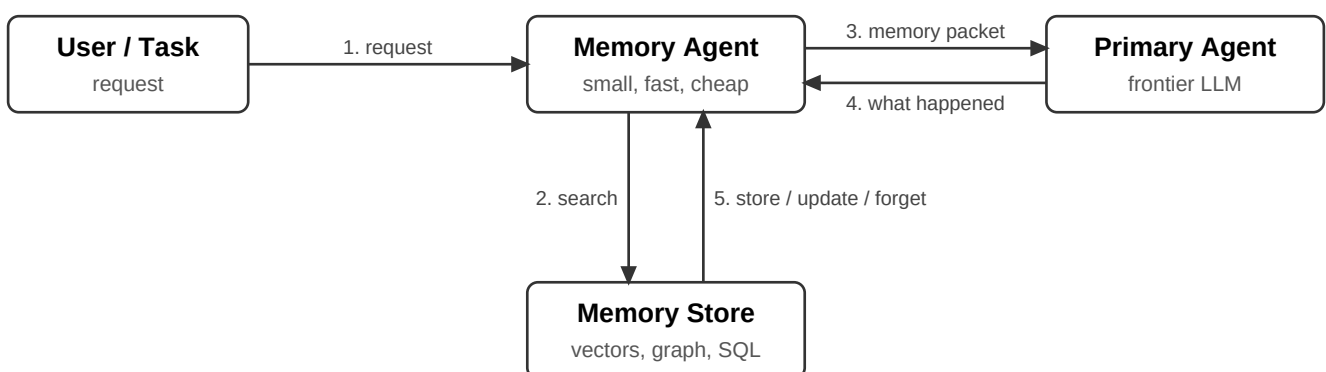
2. The problem we are solving

AI assistants build up huge histories: conversations, documents, decisions, preferences. Today the usual fix is to push as much history as possible into the model's context window. That causes:

- **High cost and slow answers.** Every token is paid for and processed on every turn.
- **Noise.** Most of the history is irrelevant, and the key detail gets buried.
- **Stale and conflicting facts.** "Uses PostgreSQL" and "moved to MongoDB" sit side by side as if both were true.
- **Endless growth.** Memory keeps growing because nothing is merged or forgotten.
- **Wasted expensive intelligence.** The biggest, costliest model spends its effort managing memory instead of solving the task.

A bigger context window is not the same as a better memory. The goal is not to make the window bigger. It is to make every token inside it matter.

3. How it works (one turn)



- **Before the answer:** the Memory Agent searches the store and sends the Primary Agent a small packet of only what matters. For "use the database we chose last month" it sends "PostgreSQL chosen, for relational constraints; task is the API layer", not a month of chat.
- **After the answer:** the Memory Agent checks what changed and updates the store. When REST changes to GraphQL, REST is marked "history" and GraphQL "current", instead of adding another note.
- **Decisions, not paragraphs:** e.g. store 0.94, importance 0.88, update memory_418, send to primary 0.96. Normal software carries them out.

4. What is new compared with existing work

Existing systems each solve one piece of memory, and most still use a large general LLM to make every memory decision.

Existing work	What it shows	What it leaves open
Memoria (2025)	Curated memory works: about 400 tokens instead of 115K, with higher accuracy than full context on LongMemEval	Uses the main LLM for extraction and summaries; recency weighting only, no learned decisions
MemoryRepository (2024)	Summarizing and forgetting under a fixed token limit	Rule-based forgetting; LLM does the summarizing; game-NPC setting
ReMAP (2025)	Self-updating facts: merge, replace, add	Single domain (travel); update logic depends on an LLM
Memory Fabric review (2026)	Maps shared and persistent memory systems	Names open problems: conflicting memories, pruning, evaluation, scale

DAMA's contribution: one dedicated, specialized Memory Agent that owns the **whole** memory lifecycle (store, retrieve, rank, update, merge, compress, forget, token budget) for a separate frontier model. It is designed as a small, cheap decision model, not another chatbot.

5. Goals

- Send far fewer tokens to the expensive model without losing answer quality.
- Keep memory accurate and current: resolve conflicts and track what changed over time.
- Stop memory growing forever by merging and forgetting safely.
- Use expensive intelligence only where deep reasoning is needed.

6. Main risk and safeguard

If the Memory Agent wrongly drops something important, the Primary Agent never sees it. DAMA therefore uses confidence scores and fallbacks: if confident, send the small packet; if unsure, retrieve more; if still unsure, ask a larger model; for critical tasks, let the Primary Agent see the raw history.

7. Status and next step

DAMA is a **proposal**. It has not been built or tested, and no DAMA results exist yet. First step: build a simple version (frontier API + small model returning JSON + vector and SQL store), then test it against full context, standard RAG and an LLM-based memory manager, keeping the same Primary Agent and measuring accuracy, tokens and latency.

Central question: can a small, specialized model learn what a much larger model should remember, retrieve, update and forget, while using less context and losing no quality?

References: Sarin et al., Memoria, arXiv 2512.12686 (2025); Zheng et al., MemoryRepository for AI NPC, IEEE Access (2024); Wang et al., ReMAP, IEEE Access (2025); Tiwari and Gupta, A memory fabric for conversational AI agents, Discover AI (2026); Wang et al., A survey on LLM-based autonomous agents, Front. Comput. Sci. (2024); Xi et al., The Rise and Potential of LLM-Based Agents, arXiv 2309.07864 (2023); Zheng et al., Machine Memory Intelligence, Engineering (2025).